

Министерство науки и высшего образования РФ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Ярославский государственный технический университет»
Кафедра «Информационные системы и технологии»

Отчет защищен
с оценкой _____
Преподаватель
_____ М. Е. Колобаев
«__» _____ 2026

РАЗРАБОТКА REST API ДЛЯ ПРЕДМЕТНОЙ ОБЛАСТИ «КЛИЕНТЫ» С ПОДДЕРЖКОЙ ОПЕРАЦИЙ GET, POST, PUT, PATCH, DELETE

Отчет о лабораторной работе №1
по курсу «Бэкенд разработка веб-приложений»

ЯГТУ 09.03.04 – 007 ЛР

Отчет выполнил
студент гр. ЦПИ-41
_____ А. В. Морев
«__» _____ 2026

Содержание

1 Цель работы	3
2 Описание предметной области	4
3 Основная часть	5
3.1 Описание и применение ручек	5
3.1.1 AuthController	5
3.1.2 UserController.....	8
3.1.3 ClientController.....	13
3.2 Проверка доступа без авторизации	20
3.3 Валидация данных.....	21
3.4 Обработка ошибок	22
3.5 Описание README	25
Приложение А. README.md	26

1 Цель работы

Целью лабораторной работы является проектирование и реализация серверного приложения, предоставляющего REST API для управления сущностями выбранной предметной области. Приложение должно корректно обрабатывать запросы клиента, возвращать HTTP-статусы, JSON-ответы и сообщения об ошибках в едином формате.

Студент должен разработать приложение, которое:

- Предоставляет REST API для работы минимум с одной основной сущностью.
- Поддерживает полный набор CRUD-операций.
- Реализует обязательную авторизацию пользователей.
- Реализует обработку ошибок в стиле REST.
- Выполняет валидацию входных данных.
- Возвращает ответы в формате JSON.
- Использует базу данных, размещённую на сервере преподавателя.

2 Описание предметной области

Выбранная предметная область – Вариант №7 «Клиенты». В ней основной сущностью является сущность «Client» со следующими полями:

- id,
- fullName,
- phone,
- email,
- discount,
- verified.

Дополнительно в серверном приложении имеется сущность «User», представляющая собой пользователей, имеющими доступ к сущностям «Client». Поля сущности «User» представлены в списке ниже:

- id,
- login,
- hashPassword.

Так как в серверном приложении используется авторизация с помощью JWT-токенов, дополнительно реализован механизм «Refresh Token», что позволяет пользователю обновить JWT-токен для дальнейшего использования приложения. Поля сущности «RefreshToken» представлены в списке ниже:

- id,
- user_id (foreign key сущности «User»),
- token,
- expires_at,
- is_revoked,
- created_at,
- replaced_by_token.

3 Основная часть

3.1 Описание и применение ручек

Доступ к ручкам осуществляется благодаря API-контроллерам, построенные вокруг конкретной задачи. В приложении описаны 3 контроллера:

- AuthController (путь: “/api/auth”),
- UserController (путь: “/api/user”),
- ClientController (путь: “/api/client”).

Ошибки в случае выполнения методов контроллера выбрасываются в ходе выполнения методов используемого в контроллере сервиса. Также контроллеры проверяют наличие авторизации у пользователя с помощью атрибута [Authorize], указанного перед классом контроллера.

3.1.1 AuthController

У AuthController имеется 3 ручки:

- Login (POST “/api/auth/login”),
- Logout (POST “/api/auth/logout”),
- Refresh (POST “/api/auth/refresh”).

Скриншоты ручек представлены на рис. 1-3 соответственно.

```
[HttpPost("login")]
[Consumes(MediaTypeNames.Application.Json)]
[ProducesResponseType(StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
[ProducesResponseType(StatusCodes.Status401Unauthorized)]
Ссылка: 0
public async Task<ActionResult<AuthResponseDto>> Login(LoginRequest request)
{
    var result = await _authService.LoginAsync(request.Login, request.Password);
    if (!result.Success)
        return Unauthorized(result);
    return Ok(result);
}
```

Рисунок 1 – Ручка «Login»

```
[HttpPost("logout")]
[ProducesResponseType(StatusCodes.Status204NoContent)]
Ссылка: 0
public async Task<IActionResult> Logout()
{
    var refreshToken = Request.Headers["X-Refresh-Token"].ToString();
    await _authService.LogoutAsync(refreshToken);

    return NoContent();
}
```

Рисунок 2 – Ручка «Logout»

```
[HttpPost("refresh")]
[Consumes(MediaTypeNames.Application.Json)]
[ProducesResponseType(StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
[ProducesResponseType(StatusCodes.Status401Unauthorized)]
Ссылка: 0
public async Task<ActionResult<AuthResponseDto>> Refresh(TokenRefreshRequest request)
{
    var result = await _authService.RefreshTokenAsync(request.RefreshToken);
    if (!result.Success)
        return Unauthorized(result);

    return Ok(result);
}
```

Рисунок 3 – Ручка «Refresh»

Отправку запросов и получение ответов реализуем с помощью файла ClientApi.http.

При отправке запроса, представленного на рис. 4, получаем ответ, представленный на рис. 5. Следующие запросы-ответы представлены аналогично (переменные, заключенные в двойные фигурные скобки, описываются в начале файла ClientApi.http).

```
POST {{ClientApi_HostAddress}}/api/auth/login
Content-Type: application/json

{
    "login": "admin",
    "password": "admin123"
}
```

Рисунок 4 – Запрос POST /api/auth/login

```

HTTP/1.1 200 OK
Connection: close
Content-Type: application/json; charset=utf-8
Date: Sun, 15 Mar 2026 09:07:11 GMT
Server: Kestrel
Transfer-Encoding: chunked

{
  "success": true,
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWxz
b2FwLm9yZy93cy8yMDA1LzA1L2lkZW50aXR5L2NsYWltcy9uYW1lYW1lbnRlbnR5ZmllciI6IjMiLCJodHRwOi8v
c2NoZW1hcy54bWxzY2FwLm9yZy93cy8yMDA1LzA1L2lkZW50aXR5L2NsYWltcy9uYW1lIjoieWRTaw4iLCJl
eHAiOiJlE3Nm1NjY1MzEsImIzcyI6Imh0dHA6Ly9sb2NhbGhvc3Q6NTI3OSIsImF1ZCI6Imh0dHA6Ly9sb2Nh
bGhvc3Q6NTI3OSJ9.ms666eCOW-6DFMKRt4-pkqaYe5JRqDSkMGYVDDAZpCg",
  "refresh_token": "kejDr+rnWTK07cfJOBqH+EQ/Ab/BdOdNU4QJQxjuCMw=",
  "expires_at": "2026-03-15T09:22:11.4222461Z",
  "error": null
}

```

Рисунок 5 – Ответ POST /api/auth/login

```

POST {{ClientApi_HostAddress}}/api/auth/logout
X-Refresh-Token: {{RefreshToken}}

```

Рисунок 6 – Запрос POST /api/auth/logout

```

HTTP/1.1 204 No Content
Connection: close
Date: Sun, 15 Mar 2026 09:09:37 GMT
Server: Kestrel

```

Рисунок 7 – Ответ POST /api/auth/logout

```

POST {{ClientApi_HostAddress}}/api/auth/refresh
Content-Type: application/json

{
  "refresh_token": "uaNZJOZ+WtGFoSDAUqky1yRyJbcJBHxOgmcfZkFTCg="
}

```

Рисунок 8 – Запрос POST /api/auth/refresh

```

HTTP/1.1 200 OK
Connection: close
Content-Type: application/json; charset=utf-8
Date: Sun, 15 Mar 2026 09:11:35 GMT
Server: Kestrel
Transfer-Encoding: chunked

{
  "success": true,
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWxz
b2FwLm9yZy93cy8yMDA1LzA1L2lkZW50aXR5L2NsYWltcy9uYW1laWRlbnRpZmllciI6IjMiLCJodHRwOi8v
c2NoZW1hcy54bWxzZb2FwLm9yZy93cy8yMDA1LzA1L2lkZW50aXR5L2NsYWltcy9uYW1lIjoieWRTaW4iLCJl
eHAiOiJlE3NzM1NjY3OTUsIm1zcyI6Imh0dHA6Ly9sb2NhbGhvc3Q6NTI3OSIsImF1ZCI6Imh0dHA6Ly9sb2Nh
bGhvc3Q6NTI3OSJ9.vWAvCUoTxDe4lUj25qq_scnNhFcdmMII7t2for-SSCc",
  "refresh_token": "+QrMSUGQMZLfxB5AJAAPffFrPUBYdJaKrByKabrBU0=",
  "expires_at": "2026-03-15T09:26:35.8755488Z",
  "error": null
}

```

Рисунок 9 – Ответ POST /api/auth/refresh

3.1.2 UserController

У UserController имеется 7 ручек:

- GetAll (GET “/api/user”),
- GetById (GET “/api/user/{id}”),
- GetByLogin (GET “/api/user/by-login?login={login}”),
- Add (POST “/api/user”),
- Update (PUT “/api/user/{id}”),
- PartialUpdate (PATCH “/api/user/{id}”),
- Delete (DELETE “/api/user/{id}”).

Скриншоты ручек представлены на рис. 10-16 соответственно.

```

[HttpGet]
[ProducesResponseType<IEnumerable<UserResponseDto>>(StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status401Unauthorized)]
Ссылка: 0
public async Task<IActionResult> GetAll()
{
    var users = await _userService.GetAllUsersAsync();
    return Ok(users);
}

```

Рисунок 10 – Ручка «GetAll» (UserController)


```

[HttpGet("{id:int}")]
[ProducesResponseType<UserResponseDto>(StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
[ProducesResponseType(StatusCodes.Status401Unauthorized)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
Ссылка: 1
public async Task<IActionResult> GetById(int id)
{
    var user = await _userService.GetUserByIdAsync(id);
    return Ok(user);
}

```

Рисунок 11 – Ручка «GetById» (UserController)

```

[HttpGet("by-login")]
[ProducesResponseType<UserResponseDto>(StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
[ProducesResponseType(StatusCodes.Status401Unauthorized)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
Ссылка: 0
public async Task<IActionResult> GetByLogin([FromQuery] string login)
{
    var user = await _userService.GetUserByLoginAsync(login);
    return Ok(user);
}

```

Рисунок 12 – Ручка «GetByLogin» (UserController)

```

[HttpPost]
[Consumes(MediaTypeNames.Application.Json)]
[ProducesResponseType<UserResponseDto>(StatusCodes.Status201Created)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
[ProducesResponseType(StatusCodes.Status401Unauthorized)]
[ProducesResponseType(StatusCodes.Status422UnprocessableEntity)]
Ссылка: 0
public async Task<IActionResult> Add(UserRequestDto userDto)
{
    var user = await _userService.AddUserAsync(userDto);
    return CreatedAtAction(nameof(GetById), new { id = user.Id }, user);
}

```

Рисунок 13 – Ручка «Add» (UserController)

```

[HttpPut("{id:int}")]
[Consumes(MediaTypeNames.Application.Json)]
[ProducesResponseType(StatusCodes.Status204NoContent)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
[ProducesResponseType(StatusCodes.Status401Unauthorized)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
[ProducesResponseType(StatusCodes.Status422UnprocessableEntity)]
Ссылка: 0
public async Task<IActionResult> Update(int id, UserRequestDto userDto)
{
    await _userService.UpdateUserAsync(id, userDto);
    return NoContent();
}

```

Рисунок 14 – Ручка «Update» (UserController)

```

[HttpPatch("{id:int}")]
[Consumes(MediaTypeNames.Application.Json)]
[ProducesResponseType(StatusCodes.Status204NoContent)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
[ProducesResponseType(StatusCodes.Status401Unauthorized)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
[ProducesResponseType(StatusCodes.Status422UnprocessableEntity)]
Ссылка: 0
public async Task<IActionResult> PartialUpdate(int id, UserPartialUpdateRequestDto userDto)
{
    await _userService.PartialUpdateUserAsync(id, userDto);
    return NoContent();
}

```

Рисунок 15 – Ручка «PartialUpdate» (UserController)

```

[HttpDelete("{id:int}")]
[ProducesResponseType(StatusCodes.Status204NoContent)]
[ProducesResponseType(StatusCodes.Status401Unauthorized)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
Ссылка: 0
public async Task<IActionResult> Delete(int id)
{
    await _userService.DeleteUserAsync(id);
    return NoContent();
}

```

Рисунок 16 – Ручка «Delete» (UserController)

Отправку запросов и получение ответов реализуем с помощью файла ClientApi.http.

При отправке запроса, представленного на рис. 17, получаем ответ, представленный на рис. 18. Следующие запросы-ответы представлены аналогично (переменные, заключенные в двойные фигурные скобки, описываются в начале файла ClientApi.http).

```

GET {{ClientApi_HostAddress}}/api/user
Authorization: Bearer {{AccessToken}}

```

Рисунок 17 – Запрос GET /api/user

```

HTTP/1.1 200 OK
Connection: close
Content-Type: application/json; charset=utf-8
Date: Sun, 15 Mar 2026 09:23:20 GMT
Server: Kestrel
Transfer-Encoding: chunked

[
  {
    "id": 3,
    "login": "admin"
  },
  {
    "id": 4,
    "login": "new_user_login"
  },
  {
    "id": 1,
    "login": "patch_new_admin"
  }
]

```

Рисунок 18 – Ответ GET /api/user

```

Send Request
GET {{ClientApi_HostAddress}}/api/user/1
Authorization: Bearer {{AccessToken}}

```

Рисунок 19 – Запрос GET /api/user/1

```

HTTP/1.1 200 OK
Connection: close
Content-Type: application/json; charset=utf-8
Date: Sun, 15 Mar 2026 09:24:03 GMT
Server: Kestrel
Transfer-Encoding: chunked

{
  "id": 1,
  "login": "patch_new_admin"
}

```

Рисунок 20 – Ответ GET /api/user/1

```

Send Request
GET {{ClientApi_HostAddress}}/api/user/by-login?login=admin
Authorization: Bearer {{AccessToken}}

```

Рисунок 21 – Запрос GET /api/user/by-login?login=admin

```

HTTP/1.1 200 OK
Connection: close
Content-Type: application/json; charset=utf-8
Date: Sun, 15 Mar 2026 09:24:41 GMT
Server: Kestrel
Transfer-Encoding: chunked

{
  "id": 3,
  "login": "admin"
}

```

Рисунок 22 – Ответ GET /api/user/by-login?login=admin

```

POST {{ClientApi_HostAddress}}/api/user
Authorization: Bearer {{AccessToken}}
Content-Type: application/json

{
  "login": "artyom_morev",
  "password": "morev_password"
}

```

Рисунок 23 – Запрос POST /api/user

```

HTTP/1.1 201 Created
Connection: close
Content-Type: application/json; charset=utf-8
Date: Sun, 15 Mar 2026 09:25:51 GMT
Server: Kestrel
Location: http://localhost:5279/api/user/5
Transfer-Encoding: chunked

{
  "id": 5,
  "login": "artyom_morev"
}

```

Рисунок 24 – Ответ POST /api/user

```

PUT {{ClientApi_HostAddress}}/api/user/5
Authorization: Bearer {{AccessToken}}
Content-Type: application/json

{
  "login": "maksim_kolobaev",
  "password": "kolobaev_password"
}

```

Рисунок 25 – Запрос PUT /api/user/5

```
HTTP/1.1 204 No Content
Connection: close
Date: Sun, 15 Mar 2026 09:27:34 GMT
Server: Kestrel
```

Рисунок 26 – Ответ PUT /api/user/5

```
PATCH {{ClientApi_HostAddress}}/api/user/5
Authorization: Bearer {{AccessToken}}
Content-Type: application/json

{
  "login": "kolobaev_is_cool"
}
```

Рисунок 27 – Запрос PATCH /api/user/5

```
HTTP/1.1 204 No Content
Connection: close
Date: Sun, 15 Mar 2026 09:28:51 GMT
Server: Kestrel
```

Рисунок 28 – Ответ PATCH /api/user/5

```
DELETE {{ClientApi_HostAddress}}/api/user/5
Authorization: Bearer {{AccessToken}}
```

Рисунок 29 – Запрос DELETE /api/user/5

```
HTTP/1.1 204 No Content
Connection: close
Date: Sun, 15 Mar 2026 09:29:39 GMT
Server: Kestrel
```

Рисунок 30 – Ответ DELETE /api/user/5

3.1.3 ClientController

У ClientController имеется 7 ручек:

- GetAll (GET “/api/client”),
- GetById (GET “/api/client/{id}”),
- GetByEmail (GET “/api/client/by-email?email={email}”),
- Add (POST “/api/client”),
- Update (PUT “/api/client/{id}”),
- PartialUpdate (PATCH “/api/client/{id}”),
- Delete (DELETE “/api/client/{id}”).

Скриншоты ручек представлены на рис. 31-37 соответственно.

```

[HttpGet]
[ProducesResponseType<IEnumerable<ClientResponseDto>>(StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status401Unauthorized)]
Ссылка: 0
public async Task<IActionResult> GetAll()
{
    var clients = await _clientService.GetAllClientsAsync();
    return Ok(clients);
}

```

Рисунок 31 – Ручка «GetAll» (ClientController)

```

[HttpGet("{id:int}")]
[ProducesResponseType<ClientResponseDto>(StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
[ProducesResponseType(StatusCodes.Status401Unauthorized)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
Ссылка: 1
public async Task<IActionResult> GetById(int id)
{
    var client = await _clientService.GetClientByIdAsync(id);
    return Ok(client);
}

```

Рисунок 32 – Ручка «GetById» (ClientController)

```

[HttpGet("by-email")]
[ProducesResponseType<ClientResponseDto>(StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
[ProducesResponseType(StatusCodes.Status401Unauthorized)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
Ссылка: 0
public async Task<IActionResult> GetByEmail([FromQuery] string email)
{
    var client = await _clientService.GetClientByEmailAsync(email);
    return Ok(client);
}

```

Рисунок 33 – Ручка «GetByEmail» (ClientController)

```

[HttpPost]
[Consumes(MediaTypeNames.Application.Json)]
[ProducesResponseType<ClientResponseDto>(StatusCodes.Status201Created)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
[ProducesResponseType(StatusCodes.Status401Unauthorized)]
[ProducesResponseType(StatusCodes.Status422UnprocessableEntity)]
Ссылка: 0
public async Task<IActionResult> Add(ClientRequestDto clientDto)
{
    var client = await _clientService.AddClientAsync(clientDto);
    return CreatedAtAction(nameof(GetById), new { id = client.Id }, client);
}

```

Рисунок 34 – Ручка «Add» (ClientController)

```

[HttpPut("{id:int}")]
[Consumes(MediaTypeNames.Application.Json)]
[ProducesResponseType(StatusCodes.Status204NoContent)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
[ProducesResponseType(StatusCodes.Status401Unauthorized)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
[ProducesResponseType(StatusCodes.Status422UnprocessableEntity)]
Ссылка: 0
public async Task<IActionResult> Update(int id, ClientRequestDto clientDto)
{
    await _clientService.UpdateClientAsync(id, clientDto);
    return NoContent();
}

```

Рисунок 35 – Ручка «Update» (ClientController)

```

[HttpPatch("{id:int}")]
[Consumes(MediaTypeNames.Application.Json)]
[ProducesResponseType(StatusCodes.Status204NoContent)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
[ProducesResponseType(StatusCodes.Status401Unauthorized)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
[ProducesResponseType(StatusCodes.Status422UnprocessableEntity)]
Ссылка: 0
public async Task<IActionResult> PartialUpdate(int id, ClientPartialUpdateRequestDto clientDto)
{
    await _clientService.PartialUpdateClientAsync(id, clientDto);
    return NoContent();
}

```

Рисунок 36 – Ручка «PartialUpdate» (ClientController)

```

[HttpDelete("{id:int}")]
[ProducesResponseType(StatusCodes.Status204NoContent)]
[ProducesResponseType(StatusCodes.Status401Unauthorized)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
Ссылка: 0
public async Task<IActionResult> Delete(int id)
{
    await _clientService.DeleteClientAsync(id);
    return NoContent();
}

```

Рисунок 37 – Ручка «Delete» (ClientController)

Отправку запросов и получение ответов реализуем с помощью файла ClientApi.http.

При отправке запроса, представленного на рис. 38, получаем ответ, представленный на рис. 39. Следующие запросы-ответы представлены аналогично (переменные, заключенные в двойные фигурные скобки, описываются в начале файла ClientApi.http).

```

GET {{ClientApi_HostAddress}}/api/client
Authorization: Bearer {{AccessToken}}

```

Рисунок 38 – Запрос GET /api/client

```

HTTP/1.1 200 OK
Connection: close
Content-Type: application/json; charset=utf-8
Date: Sun, 15 Mar 2026 10:50:23 GMT
Server: Kestrel
Transfer-Encoding: chunked

[
  {
    "id": 2,
    "full_name": "Morev Artyom Vladimirovich",
    "phone": "9966818393",
    "email": "example@example.com",
    "discount": 0.00,
    "verified": false
  }
]

```

Рисунок 39 – Ответ GET /api/client


```
POST {{ClientApi_HostAddress}}/api/client
Authorization: Bearer {{AccessToken}}
Content-Type: application/json

{
  "full_name": "Kolobaev Maksim Evgenievich",
  "phone": "9801238712",
  "email": "kolobaevma@gmail.com",
  "discount": 12.5,
  "verified": true
}
```

Рисунок 40 – Запрос POST /api/client

```
HTTP/1.1 201 Created
Connection: close
Content-Type: application/json; charset=utf-8
Date: Sun, 15 Mar 2026 10:52:22 GMT
Server: Kestrel
Location: http://localhost:5279/api/client/3
Transfer-Encoding: chunked

{
  "id": 3,
  "full_name": "Kolobaev Maksim Evgenievich",
  "phone": "9801238712",
  "email": "kolobaevma@gmail.com",
  "discount": 12.5,
  "verified": true
}
```

Рисунок 41 – Ответ POST /api/client

```
GET {{ClientApi_HostAddress}}/api/client/3
Authorization: Bearer {{AccessToken}}
```

Рисунок 42 – Запрос GET /api/client/3

```

HTTP/1.1 200 OK
Connection: close
Content-Type: application/json; charset=utf-8
Date: Sun, 15 Mar 2026 10:52:43 GMT
Server: Kestrel
Transfer-Encoding: chunked

{
  "id": 3,
  "full_name": "Kolobaev Maksim Evgenievich",
  "phone": "9801238712",
  "email": "kolobaevma@gmail.com",
  "discount": 12.50,
  "verified": true
}

```

Рисунок 43 – Ответ GET /api/client/3

```

GET {{ClientApi_HostAddress}}/api/client/by-email?email=kolobaevma@gmail.com
Authorization: Bearer {{AccessToken}}

```

Рисунок 44 – Запрос GET /api/client/by-email?email=kolobaevma@gmail.com

```

HTTP/1.1 200 OK
Connection: close
Content-Type: application/json; charset=utf-8
Date: Sun, 15 Mar 2026 10:54:00 GMT
Server: Kestrel
Transfer-Encoding: chunked

{
  "id": 3,
  "full_name": "Kolobaev Maksim Evgenievich",
  "phone": "9801238712",
  "email": "kolobaevma@gmail.com",
  "discount": 12.50,
  "verified": true
}

```

Рисунок 45 – Ответ GET /api/client/by-email?email=kolobaevma@gmail.com

```
PUT {{ClientApi_HostAddress}}/api/client/3
Authorization: Bearer {{AccessToken}}
Content-Type: application/json

{
  "full_name": "Kaspirovich Daniil Romanovich",
  "email": "kaspirovichdr@mail.ru",
  "discount": 0,
  "verified": false
}
```

Рисунок 46 – Запрос PUT /api/client/3

```
HTTP/1.1 204 No Content
Connection: close
Date: Sun, 15 Mar 2026 10:55:51 GMT
Server: Kestrel
```

Рисунок 47 – Ответ PUT /api/client/3

```
PATCH {{ClientApi_HostAddress}}/api/client/3
Authorization: Bearer {{AccessToken}}
Content-Type: application/json

{
  "phone": "9966818393"
}
```

Рисунок 48 – Запрос PATCH /api/client/3

```
HTTP/1.1 204 No Content
Connection: close
Date: Sun, 15 Mar 2026 10:56:39 GMT
Server: Kestrel
```

Рисунок 49 – Ответ PATCH /api/client/3

```
DELETE {{ClientApi_HostAddress}}/api/client/3
Authorization: Bearer {{AccessToken}}
```

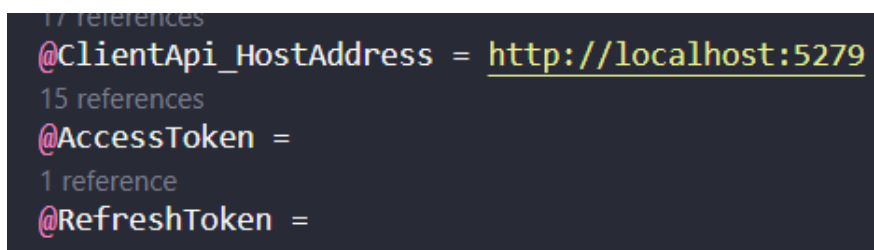
Рисунок 50 – Запрос DELETE /api/client/3

```
HTTP/1.1 204 No Content
Connection: close
Date: Sun, 15 Mar 2026 10:56:58 GMT
Server: Kestrel
```

Рисунок 51 – Ответ DELETE /api/client/3

3.2 Проверка доступа без авторизации

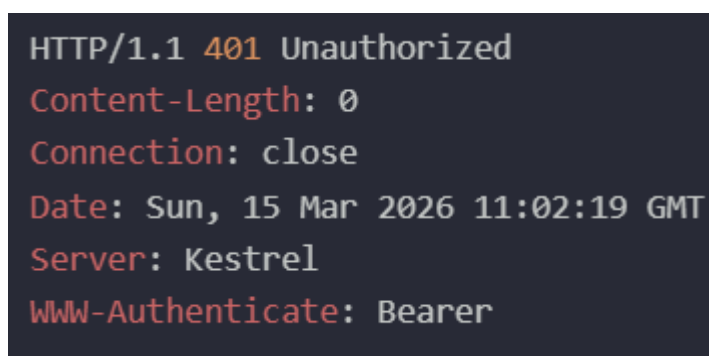
Проверим, что неавторизованный пользователь не может пользоваться ручками. Для этого в файле ClientApi.http изменим переменные так, как представлено на рис. 52.



```
17 references
@ClientApi_HostAddress = http://localhost:5279
15 references
@AccessToken =
1 reference
@RefreshToken =
```

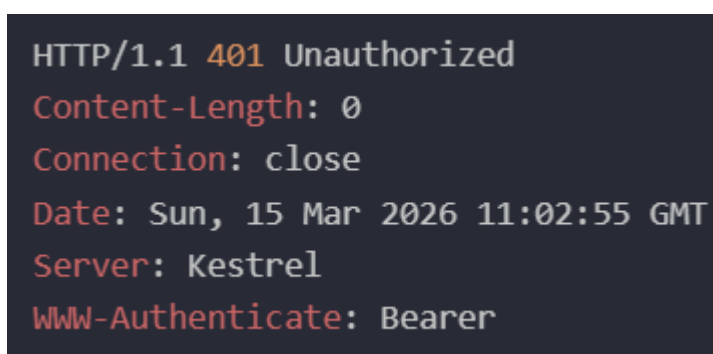
Рисунок 52 – Изменение переменных в ClientApi.http

Далее запустим по одной ручке из каждого контроллера: UserController и ClientController. Получим результаты, представленные на рис. 53-54 для каждого из контроллеров соответственно.



```
HTTP/1.1 401 Unauthorized
Content-Length: 0
Connection: close
Date: Sun, 15 Mar 2026 11:02:19 GMT
Server: Kestrel
WWW-Authenticate: Bearer
```

Рисунок 53 – Ответ при вызове ручки из UserController без авторизации



```
HTTP/1.1 401 Unauthorized
Content-Length: 0
Connection: close
Date: Sun, 15 Mar 2026 11:02:55 GMT
Server: Kestrel
WWW-Authenticate: Bearer
```

Рисунок 54 – Ответ при вызове ручки из ClientController без авторизации

Видим, что у неавторизованного пользователя нет доступа к ручкам контроллеров.

3.3 Валидация данных

Валидация данных реализована с помощью описания атрибутов валидации из библиотеки “DataAnnotations” в классах моделей и автоматическому механизму валидации данных в методах API-контроллеров благодаря установленному атрибуту [ApiController].

Например, в модели UserRequestDto, представленной на рис. 55, есть следующие атрибуты:

—Required (делает поле обязательным и устанавливает текст, возвращаемый в случае ошибки валидации);

—StringLength (устанавливает максимальную и минимальную длину поля типа «строка» и устанавливает текст, возвращаемый в случае ошибки валидации).

```

public class UserRequestDto
{
    [Required(ErrorMessage = "Login is required"),
    StringLength(50, MinimumLength = 3, ErrorMessage = "Login must be between 3 and 50 characters long")]
    Ссылка: 6
    public string Login { get; set; } = null!;

    [Required(ErrorMessage = "Password is required"),
    StringLength(256, MinimumLength = 6, ErrorMessage = "Password must be between 6 and 256 characters long")]
    Ссылка: 2
    public string Password { get; set; } = null!;
}

```

Рисунок 55 – Модель UserRequestDto

В модели ClientRequestDto, представленной на рис. 56, дополнительно имеются следующие атрибуты:

—RegularExpression (устанавливает регулярное выражение, которому должно удовлетворять значение поля);

—Range (устанавливает диапазон допустимых числовых значений для числа;

—DefaultValue (устанавливает значение по умолчанию);

—AllowedValues (устанавливает допустимые значения).

```

public class ClientRequestDto
{
    [Required(ErrorMessage = "FullName is required"), StringLength(200, ErrorMessage = "FullName must be up to 200 characters long")]
    Ссылка: 2
    public string FullName { get; set; } = null!;

    [RegularExpression(@"^\d{10}$", ErrorMessage = "Phone must contain 10 characters")]
    Ссылка: 2
    public string? Phone { get; set; }

    [Required(ErrorMessage = "Email is required"), EmailAddress(ErrorMessage = "Email must be e-mail address")]
    Ссылка: 4
    public string Email { get; set; } = null!;

    [Range(0, 100, ErrorMessage = "Discount must be between 0 and 100"), DefaultValue(0.00)]
    Ссылка: 2
    public decimal Discount { get; set; } = 0.00M;

    [AllowedValues([true, false], ErrorMessage = "Value must be one of nexts: true, false"), DefaultValue(false)]
    Ссылка: 2
    public bool Verified { get; set; } = false;
}

```

Рисунок 56 – Модель ClientRequestDto

3.4 Обработка ошибок

Обработка ошибок и выбрасываемых в ходе выполнения запросов исключений реализуется благодаря встроенному Middleware «ErrorHandlingMiddleware» в промежуточные слои ПО, который возвращает информацию об ошибках в формате «ProblemDetails», что соответствует стандарту RFC 7807 — «Problem Details for HTTP APIs».

Например, при попытке отправки запроса POST /api/user с пустым телом, получаем результат, представленный на рис. 57. При отправке того же запроса со значениями в теле, которые не удовлетворяют условиям длины строки, получаем результат, представленный на рис. 58.

```
HTTP/1.1 400 Bad Request
Connection: close
Content-Type: application/json; charset=utf-8
Date: Sun, 15 Mar 2026 11:43:09 GMT
Server: Kestrel
Transfer-Encoding: chunked

{
  "title": "Validation Error",
  "status": 400,
  "detail": "One or more validation errors occurred.",
  "instance": "/api/user",
  "errors": {
    "Login": [
      "Login is required"
    ],
    "Password": [
      "Password is required"
    ]
  },
  "traceId": "0HNK2GV0FB9HV:00000001"
}
```

Рисунок 57 – Ответ POST /api/user с пустым телом запроса

```

HTTP/1.1 400 Bad Request
Connection: close
Content-Type: application/json; charset=utf-8
Date: Sun, 15 Mar 2026 11:44:43 GMT
Server: Kestrel
Transfer-Encoding: chunked

{
  "title": "Validation Error",
  "status": 400,
  "detail": "One or more validation errors occurred.",
  "instance": "/api/user",
  "errors": {
    "Login": [
      "Login must be between 3 and 50 characters long"
    ],
    "Password": [
      "Password must be between 6 and 256 characters long"
    ]
  },
  "traceId": "0HNK2GV0FB9I0:00000001"
}

```

Рисунок 58 – Ответ POST /api/user с невалидной длиной

При отправке запроса POST /api/client с пустым телом запроса получаем результат, представленный на рис. 59.

```

HTTP/1.1 400 Bad Request
Connection: close
Content-Type: application/json; charset=utf-8
Date: Sun, 15 Mar 2026 11:46:05 GMT
Server: Kestrel
Transfer-Encoding: chunked

{
  "title": "Validation Error",
  "status": 400,
  "detail": "One or more validation errors occurred.",
  "instance": "/api/client",
  "errors": {
    "Email": [
      "Email is required"
    ],
    "FullName": [
      "FullName is required"
    ]
  },
  "traceId": "0HNK2GV0FB9I1:00000001"
}

```

Рисунок 59 – Ответ POST /api/client с пустым телом запроса

При отправке запроса POST /api/client, где в поле «email» в теле запроса содержится значение, которое уже присуще другому клиенту, получаем результат, представленный на рис. 60.

```
HTTP/1.1 422 Unprocessable Entity
Connection: close
Content-Type: application/json; charset=utf-8
Date: Sun, 15 Mar 2026 11:47:58 GMT
Server: Kestrel
Transfer-Encoding: chunked

{
  "title": "Business rule violation",
  "status": 422,
  "detail": "Client with this email already exists",
  "instance": "/api/client",
  "traceId": "0HNK2GV0FB9I4:00000001"
}
```

Рисунок 60 – Ответ POST /api/client с занятым email

При отправке запроса GET /api/client/10 при условии того, что клиента с Id = 10 не существует, получаем результат, представленный на рис. 61.

```
HTTP/1.1 404 Not Found
Connection: close
Content-Type: application/json; charset=utf-8
Date: Sun, 15 Mar 2026 11:47:02 GMT
Server: Kestrel
Transfer-Encoding: chunked

{
  "title": "Resource not found",
  "status": 404,
  "detail": "Client not found",
  "instance": "/api/client/10",
  "traceId": "0HNK2GV0FB9I2:00000001"
}
```

Рисунок 61 – Ответ GET /api/client/10 с несуществующим клиентом

При отправке запроса DELETE /api/client/3 при условии того, что клиент не существует, получаем результат, представленный на рис. 62.


```
HTTP/1.1 404 Not Found
Connection: close
Content-Type: application/json; charset=utf-8
Date: Sun, 15 Mar 2026 11:49:22 GMT
Server: Kestrel
Transfer-Encoding: chunked

{
  "title": "Resource not found",
  "status": 404,
  "detail": "Client not found",
  "instance": "/api/client/3",
  "traceId": "0HNK2GV0FB9I7:00000001"
}
```

Рисунок 62 – Ответ DELETE /api/client/3 с несуществующим клиентом

3.5 Описание README

Описание README-файла представлено в приложении А.

Приложение А. README.md

backend-development

Название проекта

****РАЗРАБОТКА REST API ДЛЯ ПРЕДМЕТНОЙ ОБЛАСТИ «КЛИЕНТЫ»
С ПОДДЕРЖКОЙ ОПЕРАЦИЙ GET, POST, PUT, PATCH, DELETE****

Описание предметной области

Выбранная предметная область – Вариант №7 «Клиенты».

В ней основной сущностью является сущность «Client» со следующими полями:

- id,
- fullName,
- phone,
- email,
- discount,
- verified.

Дополнительно в серверном приложении имеется сущность «User», представляющая собой пользователей, имеющими доступ к сущностям «Client».

Поля сущности «User» представлены в списке ниже:

- id,
- login,
- hashPassword.

Так как в серверном приложении используется авторизация с помощью JWT-токенов, дополнительно реализован механизм «Refresh Token», что позволяет пользователю обновить JWT-токен для дальнейшего использования приложения.

Поля сущности «RefreshToken» представлены в списке ниже:

- id,
- user_id (foreign key сущности «User»),
- token,
- expires_at,
- is_revoked,
- created_at,

- replaced_by_token.

стек технологий

- Язык программирования: C#
- .NET 9.0 - Модульная платформа для разработки ПО
- ASP .NET Core - Фреймворк для разработки веб-приложений
- PostgreSQL - Система управления базами данных

Как запустить проект

Для того чтобы запустить проект, на Вашем ПК должен быть установлен пакет .NET SDK, поддерживающий версию .NET 9.0.

Для запуска проекта необходимо перейти в директорию проекта и выполнить команду в любом удобном для Вас терминале: ``dotnet run``.

Также перед запуском приложения можно осуществить сборку проекта с помощью команды ``dotnet build``.

В случае, если Вам необходимо поменять порт, на котором будет запускаться веб-приложение, Вам необходимо открыть файл ``launchSettings.json`` в папке ``Properties`` и изменить порт в поле ``applicationUrl``.

Описание маршрутов API

Документация Swagger

Маршруты API можно просмотреть с помощью Swagger UI, предварительно запустив проект и перейти по пути ``http://localhost:5279/swagger``.

Текстовое описание

Доступ к ручкам осуществляется благодаря API-контроллерам, построенным вокруг конкретной задачи/сущности. В приложении описаны 3 контроллера:

- AuthController (путь: ``"/api/auth``),
- UserController (путь: ``"/api/user``),
- ClientController (путь: ``"/api/client``).

AuthController

У AuthController имеется 3 ручки:

- Login (POST “/api/auth/login”),
- Logout (POST “/api/auth/logout”),
- Refresh (POST “/api/auth/refresh”).

UserController

У UserController имеется 7 ручек:

- GetAll (GET “/api/user”),
- GetById (GET “/api/user/{id}”),
- GetByLogin (GET “/api/user/by-login?login={login}”),
- Add (POST “/api/user”),
- Update (PUT “/api/user/{id}”),
- PartialUpdate (PATCH “/api/user/{id}”),
- Delete (DELETE “/api/user/{id}”).

ClientController

У ClientController имеется 7 ручек:

- GetAll (GET “/api/client”),
- GetById (GET “/api/client/{id}”),
- GetByEmail (GET “/api/client/by-email?email={email}”),
- Add (POST “/api/client”),
- Update (PUT “/api/client/{id}”),
- PartialUpdate (PATCH “/api/client/{id}”),
- Delete (DELETE “/api/client/{id}”).

Примеры запросов и ответов

Документация Swagger

Примеры запросов и ответов можно просмотреть с помощью Swagger UI во вкладке конкретного метода, предварительно запустив проект и перейдя по пути `http://localhost:5279/swagger`.

Примеры ошибок

Примеры ошибок при выполнении запросов можно просмотреть с помощью Swagger UI во вкладке конкретного метода, в разделе **“Responses”**, предварительно запустив проект и перейдя по пути `http://localhost:5279/swagger`.

Описание структуры проекта

Описание структуры проекта можно посмотреть в [репозитории GitHub](<https://github.com/Mordahaoff/backend-development/tree/dev>).